Your first chat with Claude

Create an application in a single Python file for the given prompt: The university is concerned about student safety on campus, especially during evening hours. Some students feel uneasy walking alone between buildings or across the campus. To address this, the university is planning a mobile application that connects students who need someone to accompany them with other students willing to help. Imagine a student leaving the library late at night who wants to walk safely to the dorms. They can submit a request through the app, indicating where they are, where they want to go, and the time they need help. On the other side, students volunteering as “helpers” can see requests and respond to them. Since many students may request assistance at similar times, the system needs to manage multiple incoming requests while ensuring that helpers are not overwhelmed or assigned in conflicting ways. In some situations, there may be no helpers available at all, while in others, several helpers may respond to the same request. The system must ensure that helpers receive only the information necessary to assist the student safely, without exposing sensitive details about the student. At the same time, helpers should be able to communicate with students quickly, safely, and reliably. The university expects that the system remains responsive even when many students are using it at the same time.

Do not: make more than one file, include any testing, go beyond the scope of the description.

Show less

Viewed a file, created a file, read a file >

Run it with:

```bash
python safewalk.py
# then open http://localhost:8000
```

No dependencies — pure Python standard library only.

Here's how the design maps to the problem description:

Privacy by design — `WalkRequest.public_view()` strips all private student identity before anything is ever sent to a helper. Only a chosen alias, route, and timing are exposed.

Concurrency — `ThreadingMixIn` gives each HTTP connection its own daemon thread, so long-lived SSE streams never block regular API calls. All shared state mutations go through a single `threading.Lock`.

Request lifecycle — `PENDING → MATCHED → COMPLETED / CANCELLED`. Once a helper accepts, the request is locked so no second helper can claim it (a `409 Conflict` is returned to racing volunteers).

Helper availability — Helpers carry an `is_available` flag and `assigned_request`. They cannot accept a second request while busy, preventing conflicting assignments.

Real-time communication — A Server-Sent Events endpoint (`/api/events`) broadcasts `request_created`, `request_updated`, and `chat_message` events to all connected clients instantly, so the UI stays live without polling.

No-helper scenario — Requests stay `PENDING` indefinitely and remain visible on all connected clients until a helper appears or the student cancels.